

CHAPTER 2

JSP Elements

A JSP page is made out of a page template, which consists of HTML code and JSP elements such as *scripting* elements, *directive* elements, and *action* elements. In the previous chapter, after explaining how to install Java, Tomcat, and Eclipse, I introduced you to JSP and explained JSP's role within web applications. In this chapter, I'll describe in detail the first two types of JSP elements. For the action elements, refer to the next chapters.

Introduction

Scripting elements consist of code delimited by particular sequences of characters. The scriptlets, which you encountered in the examples in Chapter 1 and delimited by the pair `<%` and `%>`, are one of the three possible types of scripting elements. The other two are declarations and expressions.

All scripting elements are Java fragments capable of manipulating Java objects, invoking their methods, and catching Java exceptions. They can send data to the output, and they execute when the page is requested. Tomcat defines several *implicit objects*: *application*, *config*, *exception*, *out*, *pageContext*, *request*, *response*, and *session*.

Directive elements are messages to the JSP container (i.e., Tomcat). Their purpose is to provide information on the page itself necessary for its translation. As they have no association with each individual request, directive elements do not output any text to the HTML response.

The first line of the `hello.jsp` example was a directive:

```
<%@page language="java" contentType="text/html"%>
```

Besides `page`, the other directives available in JSP pages are `include` and `taglib`.

Action elements specify activities that, like the scripting elements, need to be performed when the page is requested, because their purpose is precisely to encapsulate activities that Tomcat performs when handling an HTTP request from a client. Action elements can use, modify, and/or create objects, and they may affect the way data is sent

to the output. There are more than a dozen standard actions: `attribute`, `body`, `element`, `fallback`, `forward`, `getProperty`, `include`, `param`, `params`, `plugin`, `setProperty`, `text`, and `useBean`. For example, the following action element includes in a JSP page the output of another page:

```
<jsp:include page="another.jsp"/>
```

In addition to the standard action elements, JSP also provides a mechanism that lets you define custom actions, in which a prefix of your choice replaces the prefix `jsp` of the standard actions. The *tag extension mechanism* lets you create libraries of custom actions, which you can then use in all your applications. Several custom actions became so popular within the programming community that Sun Microsystems (now Oracle) decided to standardize them. The result is JSTL, the JSP Standard Tag Library.

The *Expression Language* (EL) is an additional JSP component that provides easy access to external objects (i.e., Java beans). EL was introduced in JSP 2.0 as an alternative to the scripting elements, but you can also use EL and scripting elements together.

In the next sections, I will first go through the scripting elements, because they are easier to understand and you can use them to glue together the rest. Then, I will describe the implicit objects and the directives. To help you find the correct examples in the software package for this chapter, I divided them in folders named according to the section title and the functionality tested (e.g., request object - body).

Scripting Elements and Java

Scripting elements let you embed Java code in an HTML page. Every Java executable—whether it's a free-standing program running directly within a runtime environment or a servlet executing in a container such as Tomcat—boils down to instantiating classes into objects and executing their methods. This might not be so apparent with JSP, since Tomcat wraps every JSP page into a class of type `Servlet` behind the scenes, but it still applies.

A Java method consists of a sequence of operations—executed when the method is called—with the scope to instantiate objects, allocate memory for variables, calculate expressions, perform assignments, or execute other specific instructions.

In this section, I'll summarize the syntax of Java while keeping JSP in mind.

Scriptlets

A scriptlet is a block of Java code enclosed between `<%` and `%>`. For example, this code includes two scriptlets that let you switch an HTML element on or off depending on a condition:

```
<% if (condition) { %>
<p>This is only shown if the condition is satisfied</p>
<% } %>
```

Expressions

An expression scripting element inserts into the page the result of a Java expression enclosed in the pair `<%=` and `%>`. For example, in the following snippet of code, the expression scripting element inserts the current date into the generated HTML page:

```
<%@page import="java.util.Date"%>
Server date and time: <%=new Date()%>
```

You can use within an expression scripting element any Java expression, provided it results in a value. In practice, it means that every Java expression will do, except the execution of a method of type `void`. For example, `<%= (condition) ? "yes" : "no"%>` is valid, because it calculates to a string. You would obtain the same output with the scriptlet `<%if (condition) out.print("yes") else out.print("no");%>`.

Note that an expression is not a Java statement. Accordingly, it has no semicolon at the end.

Declarations

A declaration scripting element is a Java variable declaration enclosed between `<%!` and `%>`. It results in an instance variable shared by all requests for the same page.

Data Types and Variables

Java makes available primitive data types similar to the basic types of C/C++ (see Table 2-1).

Table 2-1. *Java Data types*

Name	Class	Description
byte	Byte	1-byte signed integer (−128 to +127)
short	Short	2-byte signed integer (−32,768 to +32,767)
int	Integer	4-byte signed integer (−2,147,483,648 to +2,147,483,647)
Long	Long	8-byte signed integer (approximately -10^{19} to $+10^{19}$)
Float	Float	32-bit signed floating point (8-bit exponent, 23-bit precision)
Double	Double	64-bit signed floating point (11-bit exponent, 52-bit precision)
Char	Character	16-bit unsigned Unicode
boolean	Boolean	Either true or false

The second column of Table 2-1 gives you the names of the so-called *wrapper classes* that Java makes available for each primitive type. These classes provide some useful static methods to manipulate numbers. For example, `Integer.parseInt(String s, int radix)` interprets a string as a number in the base set by the second argument and returns it as an `int` value (e.g., `Integer.parseInt("12", 16)` and `Integer.parseInt("10010", 2)` both return 18).

Programs in Java can be platform-independent because all platform dependencies are “hidden” inside libraries. The wrapper classes I just mentioned are in the `java.lang` library, together with dozens of other general classes such as `String` and `Math`. You can find the full documentation of the Java 13 platform at <https://docs.oracle.com/en/java/javase/13/docs/api/index.html>.

Here are some examples of how you can declare variables and initialize them:

```
String aString = "abcdxyz";
int k = aString.length(); // k is then set to 7
char c = aString.charAt(4); // c is set to 'x'
static final String NAME = "John Doe";
```

The `final` keyword in the last example of declarations makes the variable unchangeable. This is how you define constants in Java. The `static` keyword indicates that a variable is to be shared by all objects within the same application that are instantiated from the class.

The use of static variables in JSP requires some further comment. In JSP, you can declare variables in three ways:

```
<% int k = 0; %>
<%! int k = 0; %>
<%! static int k = 0; %>
```

The first declaration means that a new variable is created for each incoming HTTP client request; the second one means that a new variable is created for each new instance of the servlet; and the third one means that the variable is shared among all instances of the servlet.

Tomcat converts each JSP page into a subclass of the HTTP Servlet class (`javax.servlet.http.HttpServlet`). Normally, Tomcat instantiates each one of these classes only once and then creates a Java thread for each incoming request. It then executes the same servlet object within each thread. If the application runs on a distributed environment or for high numbers of requests, Tomcat can instantiate the same servlet more than once. Therefore, only the third declaration guarantees that the variable will be shared among all requests.

Tomcat keeps the servlet code buried deep in the folder named `work`. For example, the servlet generated from `webapps\ROOT\test\a.jsp` is in `work\Catalina\localhost\_\org\apache\jsp\test\` and is named `a_jsp.java`.

You're free to name your variables as you like, though your case-sensitive string of characters must begin with a letter, a dollar, or an underscore and not contain a space. That said, be aware that the following keywords are reserved and will cause a compilation error: `abstract`, `assert`, `boolean`, `break`, `byte`, `case`, `catch`, `char`, `class`, `const`, `continue`, `default`, `do`, `double`, `else`, `enum`, `extends`, `final`, `finally`, `float`, `for`, `goto`, `if`, `implements`, `import`, `instanceof`, `int`, `interface`, `long`, `native`, `new`, `package`, `private`, `protected`, `public`, `return`, `short`, `static`, `strictfp`, `super`, `switch`, `synchronized`, `this`, `throw`, `throws`, `transient`, `try`, `void`, `volatile`, and `while`. Whenever possible, use capital letters for constants. It is not necessary, but it makes the code more readable and is a well-established coding practice.

To use special characters within a string, you need to escape them with a backslash, as shown in Table 2-2. With `\u` followed by up to four hexadecimal digits, you can specify any Unicode character. For example, you can enter the Greek capital letter delta as `\u0394`.

Table 2-2. *Escaped Special Characters*

Character	Escaped
backslash	<code>\\</code>
backspace	<code>\b</code>
carriage return	<code>\r</code>
double quote	<code>\"</code>
form feed	<code>\f</code>
line feed	<code>\n</code>
single quote	<code>\'</code>
tab	<code>\t</code>

Objects and Arrays

To create an object of a certain type (i.e., to instantiate a class) and allocate memory, use the keyword `new`. In the following example, a new Java object of type `String` is created:

```
String newString= new String();
```

This creates an object of type `Integer` with value 55.

You can have arrays of any object type or primitive data type, as in the following examples of array declarations:

```
int[] intArray1;
int[] intArray2 = {10, 100, 1000};
String[] stringArray = {"a", "bb"};
```

`intArray1` is null; `intArray2` is an array of length three containing 10, 100, and 1000; and `stringArray` is an array of length two containing the strings "a" and "bb".

Although arrays look special, they're actually just objects and treated like that. Therefore, you can initialize them with `new`. For example, the following line of code declares an integer array with ten elements, each initialized to zero:

```
int[] array = new int[10];
```

A two-dimensional table is an array in which each element object is itself an array. This is *not* like in C, where a single block of memory contains all elements of multidimensional tables. For example, this line of code represents a table of two rows, but the first row has three elements, while the second one has only two:

```
int[][] table1 = {{11, 12, 13}, {21, 22}};
```

If you define something like this:

```
int[][] table = new int[2][3];
```

you have a table with two rows and three columns, with all elements initialized to zero.

Operators, Assignments, and Comparisons

There are no surprises with the binary operators—that is, the operators that require two operands. They include the expected addition, subtraction, multiplication, division, and modulus (i.e., the remainder of an integer division) operators. When applied to string, the addition operator concatenates them.

Besides the normal assignment operator represented by the equal sign, there is also an assignment operator for each binary operator. For example, the following line of code means that you take the current value of the variable `a`, add to it `b`, and store it back into `a`:

```
a += b; // same as a = a + b;
```

The most commonly used unary operators (i.e., operators that require a single operand) include the minus sign, which changes the sign of what follows, and the increment and decrement operators:

```
a = -b;
a++; // same as a += 1;
a--; // same as a -= 1;
```

You can assign the value of an expression of one type to a variable of another type, but with some restrictions. With numeric types, you can only assign values to variables that are of the same type or “larger.” For example, you can assign an `int` value to a variable of type `long`, but to assign a `long` value to an `int` variable, you’d have to *typecast* (i.e., downcast) the value, as in `int iVar = (int)1234567L;`. Be careful with that, because you might lose precision when downcasting floating-point numbers!

You can assign objects to variables of other types, but only if the type of the variable is a superclass of the class from which you instantiated the object. Similarly to the downcasting of numeric types, you can typecast a value of a superclass into a variable of a subclass type.

Comparison operators are straightforward when applied to primitive data types. You have `==` to check for equality, `!=` to check for inequality, `>` to check for “greater than,” `>=` to check for “greater than or equal to,” `<` to check for “less than,” and `<=` to check for “less than or equal to.” Nothing surprising there. However, you have to be careful when you make comparisons between objects, as the following example illustrates:

```
String s1 = "abc";
String s2 = "abc";
String s3 = "abcd".substring(0,3);
boolean b1 = (s1 == "abc"); // parentheses not needed but nice!
boolean b2 = (s1 == s2);
boolean b3 = (s1 == s3);
```

As perhaps you expected, `b1` and `b2` turn out to be `true`, but `b3` is `false`, although `s3` was set to `"abc"`! The problem is that comparison operators don’t look inside the objects. They only check whether the objects are *the same instance* of a class, not whether they hold the same value. Therefore, as long as you shift around the `"abc"` string, the compiler keeps referring to the same instance of a literal string, and everything behaves as expected. However, when you create a different instance of `"abc"`, the check for equality fails. The lesson to be learned is that if you want to compare the content of objects, you have to use the `equals` method. In this example, `s1.equals(s3)` would have returned `true`.

For objects, you also have the comparison operator `instanceof`, which isn’t available for primitive data types like `int`. For example, `("abc" instanceof String)` calculates to `true`. Be aware that an object isn’t only an instance of the class it was instantiated from, but it’s also an instance of all its superclasses up to and including `Object`, which is the

superclass of all classes. It makes sense: a `String` is also an `Object`, even if the reverse often is not true.

With `&&` for *logical and*, `||` for *logical or*, and `!` for *logical not*, you can concatenate comparisons to form more complex conditions. For example, `((a1 == a2) && !(b1 || b2))` calculates to `true` only if `a1` equals `a2` and both boolean variables `b1` and `b2` are `false`.

Selections

The following statement assigns to the string variable `s` a different string depending on a condition:

```
if (a == 1) {
    s = "yes";
}
else {
    s = "no";
}
```

You can omit the `else` part.

You could have achieved an identical result with a conditional expression and a single assignment:

```
String s = (a == 1) ? "yes" : "no";
```

You could also achieve the same result with the following code:

```
switch(a) {
    case 1:
        s = "yes";
        break;
    default:
        s = "no";
        break;
}
```

Obviously, the switch statement is only useful when there are more than just two alternatives. For example, instead of having a chain of if/else statements, as in the following example:

```
if (expression == 3) {...}
else if (expression == 10) {...}
else {...}
```

you would gain both in clarity and in conciseness with

```
switch (expression) {
  case (3): ... break;
  case (10): ... break;
  default: ... break;
}
```

At the very least, you'll calculate the expression only once. Note that if you omit a break, execution continues to the following case.

The switch variable can be of type String. Therefore, you can write switches like the following one:

```
String yn;
...
switch (yn) {
  case ("y"): /* handle the yes case */ break;
  case ("n"): /* handle the no case */ break;
  default: /* is something fishy going on? */ break;
}
```

Iterations

This statement repeatedly executes the *statements* with increasing values of *k*, beginning from *init-value*:

```
for (int k = init-value; k < limit; k++) { statements; }
```

The general format is

```
for (initial-assignment; end-condition; iteration-expression) { statements; }
```

The *initial-assignment* is executed only once, before entering the loop. The *statements* are then repeatedly executed as long as the *end-condition* is satisfied. As the *end-condition* is checked before executing the *statements*, they are not executed at all if the *end-condition* is false from the beginning. The *iteration-expression* is executed at the end of each iteration, before the *end-condition* is checked to see whether the loop should be reentered for a new iteration.

You can omit either the *initial-assignment* or the *iteration-expression*. If you omit both, you should replace the for loop with a while loop. The following two lines are equivalent:

```
while (end-condition) { statements; }
for (;end-condition;) { statements; }
```

The do-while statement is an alternative to the while loop:

```
do { statements; } while (end-condition);
```

The do-while statement checks the *end-condition* at the end of an iteration instead of at the beginning, like the for and while loops do. As a result, the statements inside a do-while loop are always executed at least once, even when the *end-condition* is false from the beginning.

The iteration statements described so far are identical to those of C, but Java also supports a variant of the for loop tailored to make the handling of collections easier. Suppose you need a method that produces a concatenation of a set of strings. It might look like this:

```
String concatenate(Set<String> ss) {
    String conc = "";
    Iterator<String> iter = ss.iterator();
    while (iter.hasNext()) {
        conc += iter.next();
    }
    return conc;
}
```

With the Java for-each variant of the for loop, you can drop the definition of the iterator and write clearer code:

```
String concatenate(Set<String> ss) {  
    String conc = "";  
    for (String s : ss) {  
        conc += s;  
    }  
    return conc;  
}
```

Implicit Objects

The most commonly used implicit objects defined by Tomcat are out and request, followed by application and session. But I will go through them in alphabetical order, for ease of reference.

Whether you create objects within JSP pages or Tomcat implicitly creates them for you, you cannot use them properly unless you know in which scope they are available. There are four possible scopes. In order of increasing generality, they are *page*, *request*, *session*, and *application*. You will learn more about them in the following sections.

In general, if you are not sure what class a particular object instantiates, you can always display its name with the following expression:

```
<%=the_mysterious_object.getClass().getName()%>
```

The application Object

The application object is an instance of the class `org.apache.catalina.core.ApplicationContextFacade`, which Tomcat defines to implement the interface `javax.servlet.ServletContext`. It provides access to the resources shared within the web application. For example, by adding an attribute (which can be an object of any type) to application, you can ensure that all JSP files that make up your web application have access to it.

Example: Using an Attribute to Enable and Disable Conditional Code

One of the advantages of using JSP is that the web server doesn't need to reinterpret the source file of a page every time a client requests that page. The JSP container translates each JSP page into a Java file and compiles it into a class, but this only happens when you update the JSP source. You might like to be able to switch on or off some particular functionality for debugging or other purposes, without having to edit one or more files and force Tomcat to recompile them when you flip the switch. To achieve this result, you only need to wrap the functionality in question inside a conditional statement, as the following one:

```
if (application.getAttribute("do_it") != null) {
    /* ...place your "switchable" functionality here... */
}
```

You also need to include two small JSP pages in your application: the first one to set the attribute `do_it` (see Listing 2-1) and the second one to remove it (see Listing 2-2).

Listing 2-1. `do_it.jsp`

```
<%@page language="java" contentType="text/html"%>
<html><head><title>Conditional code ON</title></head>
<body>Conditional code
<%
    application.setAttribute("do_it", "");
    if (application.getAttribute("do_it") == null) out.print("not");
    %>
enabled</body></html>
```

Listing 2-2. `do_it_not.jsp`

```
<%@page language="java" contentType="text/html"%>
<html><head><title>Conditional code OFF</title></head>
<body>Conditional code
```

```

<%
    application.removeAttribute("do_it");
    if (application.getAttribute("do_it") == null) out.print("not");
    %>
enabled</body></html>

```

Create new jsps in the same test Project folder created in Chapter 1 and copy the code or place them in the WebContent folder. At last, start Tomcat in Eclipse and type in your browser the URL when you want to enable the conditional code:

`http://localhost:8080/test/do_it.jsp`

Until you disable it by typing the URL

`http://localhost:8080/test/do_it_not.jsp`

or by restarting Tomcat, the conditional code will remain enabled in all pages of your application. Notice that in the example `do_it.jsp` only sets the attribute `do_it` to an empty string, but you can also define different values to have a finer selection of code to be activated.

Note that you can use the same mechanism to switch on and off HTML code.

The config Object

The config object is an instance of the `org.apache.catalina.core.StandardWrapperFacade` class, which Tomcat defines to implement the interface `javax.servlet.ServletConfig`. Tomcat uses this object to pass information to the servlets.

The following config method is the only one you might ever use; its use is trivial:

```
config.getServletName()
```

The exception Object

The exception object is an instance of a subclass of `Throwable` (e.g., `java.lang.NullPointerException`) and is only available in error pages.

Listing 2-3 shows you two methods to send the stack trace to the output. The first one, using `getStackTrace`, gives you access to each trace element as an object of type `java.lang.StackTraceElement`, which you can then analyze with methods such as `getClassName`, `getFileName`, `getLineNumber`, and `getMethodName`.

Listing 2-3. stack_trace.jsp

```

<%@page language="java" contentType="text/html"%>
<%@page import="java.util.*, java.io.*"%>
<%@page isErrorPage="true"%>
<html><head><title>Print stack trace</title></head><body>
From exception.getStackTrace():<br/>
<pre><%
    StackTraceElement[] trace = exception.getStackTrace();
    for (int k = 0; k < trace.length; k++) {
        out.println(trace[k]);
    }
%></pre>
Printed with exception.printStackTrace(new PrintWriter(out)):
<pre><%
    exception.printStackTrace(new PrintWriter(out));
%></pre>
</body></html>

```

Notice the directive `<%@page isErrorPage="true"%>`, without which the implicit object `exception` is not defined.

If you execute this page as if it were a normal page, you will get a `NullPointerException`. Listing 2-4 shows a simple example of how you can use an error page.

Listing 2-4. cause_exception.jsp

```

<%@page language="java" contentType="text/html"%>
<%@page errorPage="stack_trace.jsp"%>
<html><head><title>Cause null pointer exception</title></head><body>
<%
    String a = request.getParameter("notThere");
    int len = a.length(); // causes a null pointer exception
%>
</body></html>

```

Notice the `<%@page errorPage="stack_trace.jsp"%>` directive, which links the error page of Listing 2-4 to the occurrence of exceptions. To cause a `NullPointerException`, the page requests a parameter that doesn't exist and then accesses it. If you use `try/catch` to trap the exception, obviously the error page is not executed.

To see the two pages in action, create new jsps in the same test Project folder created in Chapter 1 and copy the code or place them in the WebContent folder. At last, start Tomcat in Eclipse and type in a browser the URL

`http://localhost:8080/test/cause_exception.jsp`.

The out Object

You use the `out` object in JSP as you use the `System.out` object in Java: to write to the standard output. The standard output for a JSP page is the body of the HTML response sent back to the client. Therefore, the scriptlet `<%out.print(expression);%>` causes the result of the expression to be displayed in the client's browser. You can achieve the same result by simply typing `<%=expression%>`.

Keep in mind that whatever you write in a JSP page outside scriptlets and other JSP elements is sent to the output anyway. Therefore, the following three lines have exactly the same effect on the response:

```
<% out.print("abc"); %>
<%= "abc"%>
abc
```

Clearly, it makes no sense to use the first two formats when you need to write literal values. To decide whether to use a scriptlet delimited by `<%..%>` or an expression delimited by `<%=..%>`, you should look at the surrounding code and decide what makes it as easy to read as possible.

The most useful methods of the object `out` are `print` and `println`. The only difference between the two is that `println` appends a newline character to the output. As an argument, both methods accept a string or any other primitive type variable. In the following example, the `int` value stored in `intVar` is automatically converted to a string:

```
out.print("a string" + intVar + obj.methodReturningString() + ".");
```


Incidentally, you could use either of the following two methods to do the conversion manually:

```
String s = Integer.toString(intVar);
String s = "" + intVar;
```

Be aware that if you try to print an object or an array by sticking its name into a print statement, you *won't* necessarily see its content in the output. If the object doesn't support a `toString()` method, you'll see a mysterious string representing the reference to the object.

Most manuals state that `out` is an instance of the `javax.servlet.jsp.JspWriter` class, which you can use to write into the response. This is not entirely correct, because `JspWriter` is an abstract class, and as such, it cannot be instantiated. In reality, `out` is an instance of the nonabstract class `org.apache.jasper.runtime.JspWriterImpl`, which extends `JspWriter`. Tomcat defines `JspWriterImpl` precisely to implement the `JspWriter` methods. For all practical purposes, this is inconsequential to you, but some of you sharp-eyed readers might have thought that I was talking about instantiating an abstract class. It usually pays to be precise.

The `JspWriter` class includes the definition of a handful of fields. You won't need them, but mentioning them gives me the opportunity to give you some useful information.

The `autoFlush` field tells you whether the `JspWriter` is flushed automatically when its buffer fills up or whether an `IOException` is thrown upon overflow. The default for `out` is `true`, which means that Tomcat will send a partial response to the client if the buffer fills up. You can set it to `false` with the directive `<%@page autoFlush="false"%>`, and you should do so if you expect the client to be an application. Sending the response in “chunks” is perfectly OK when the client is a browser, but an application will probably expect the response in a single block. If you expect the client to be an application and set `autoFlush` to `false`, you should also use `<%@page buffer="size-in-kb"%>`, to ensure that the output buffer is large enough to store your largest response. The field `autoFlush` is protected, but you can obtain its value with the `isAutoFlush` method.

The `bufferSize` field is the size in bytes of the output buffer. The default for `out` is 8192 bytes. It's a protected field, but you can obtain its value with the `getBufferSize` method.

There are also three constant integer fields (`DEFAULT_BUFFER`, `NO_BUFFER`, and `UNBOUNDED_BUFFER` of type `public static final int`), but you can safely ignore them. Just for the record, they're respectively used to test whether the `JspWriter` is buffered (and uses the default buffer size), isn't buffered, or is buffered with an unbounded buffer.

Besides the fact that you have no variable or attribute to check against these values, you're in any case well served by the `getBufferSize` method (which returns 0 if the output is not buffered).

You've already seen in several examples that you can use `print` and `println` to write to the output buffer. As an argument, you can use any of the eight primitive data types of Java (`boolean`, `char`, `byte`, `short`, `int`, `long`, `float`, and `double`), an array of characters (`char[]`), an object (`java.lang.Object`), or a string (`java.lang.String`). In practice, you'll usually use a `String` argument, as in the following example:

```
out.print("fun(" + arg + ") = " + fun(arg));
```

Here, `fun(arg)` is executed, and both `arg` and the value returned by `fun(arg)` are automatically converted to strings to be concatenated with the rest.

The `write` method, inherited from `java.io.Writer`, sends a portion of an array of characters or of a string to the output. For example, if `cbuf` is a variable of type `char[]`, `out.write(cbuf, offs, len)` will write a portion of `cbuf`, with `offs` being the offset of the first character and `len` being the number of characters to be copied. You could achieve the same result by extracting a part of the array with the following code and then printing it with `print`:

```
char[] portion = java.util.Arrays.copyOfRange(cbuf, offs, offs+len-1)
```

However, it would be less efficient, because first you would be copying a portion of the original array—an operation you don't need when using `write`.

You're not likely to use any of the other methods, and you should definitely avoid using `close`, which closes the output stream. Tomcat closes the stream when it is safe to do so, and you don't want to fiddle with it.

The pageContext Object

Most manuals state that `pageContext` is an instance of the `javax.servlet.jsp.PageContext` class to access all objects and attributes of a JSP page. Similar to what I said concerning `JspWriter`, this is only partly true, because this class, like `JspWriter`, is also abstract. In reality, `pageContext` is an instance of the nonabstract class `org.apache.jasper.runtime.PageContextImpl`, which extends `PageContext`.

The `PageContext` class defines several fields, including `PAGE_SCOPE`, `REQUEST_SCOPE`, `SESSION_SCOPE`, and `APPLICATION_SCOPE`, which identify the four possible scopes.

It also supports more than 40 methods, about half of which are inherited from the `javax.servlet.jsp.JspContext` class.

You have to pay particular attention when using the `removeAttribute` method, which accepts either one or two arguments.

For example, `pageContext.removeAttribute("attrName")` removes the attribute from *all* scopes, while the following code only removes it from the page scope:

```
pageContext.removeAttribute("attrName", PAGE_SCOPE)
```

The request Object

The request variable gives you access within your JSP page to the HTTP request sent to it by the client. It's an instance of the `org.apache.catalina.connector.RequestFacade` class, which Tomcat defines to implement the `javax.servlet.http.HttpServletRequest` and `javax.servlet.ServletRequest` interfaces.

More on Request Parameters and Client Info

In Chapter 1, you have already seen how to list all the parameters of a request. When accessing individual parameters by name, you should use some caution. Typically, you do it with the following line of code:

```
String myPar = request.getParameter("par-name");
```

And then, you do something with the parameter only if it exists—that is, if `getParameter` returns a non-null value:

```
if (par != null) { ...
```

Note that in the request generated by a URL like this:

```
http://localhost:8080/my_page.jsp?aaa&bbb=&ccc=3
```

the parameters `aaa` and `bbb` exist but are set to the empty string. Therefore, `getParameter` does *not* return null for them.

As you already saw in Chapter 1, the request can include more than one value associated with the same parameter. For example, the following URL generates a request with three values for the parameter `aaa`:

```
http://localhost:8080/my_page.jsp?aaa&aaa=4&aaa=7
```

If you execute `getParameter`, you only get the first value, which is the empty string in the example. If you want to get them all, you have to use a different method:

```
String[] ppar = request.getParameterValues("par-name");
```

and this returns an array of strings. To check that the parameter has actually been set only once and to something other than the empty string, you might then perform the following test:

```
if (ppar != null && ppar.length == 1 && ppar[0].length() > 0) { ...
```

In Chapter 1, you also saw how to determine the type of browser that sent the request. Another useful piece of information you can get about the client is its preferred locale. For example, the following line of code could set the variable `clientLocale` to the string `"en_US"`:

```
String clientLocale = request.getLocale().toString();
```

But if the viewer were in a country where a language other than English is spoken, you might get other locales (e.g., `"de_DE"` for German). If you had a multilingual site, the locale would tell you the working language of your user. You could check whether you support it and, if you do, set it as a default for the response.

The `getRemoteHost` method, which returns the client's hostname (or that of its proxy server), could be useful in a similar way, because you could look at the string after the last dot to identify foreign domain names (e.g., `it` for Italy). Unfortunately, in many cases, the remote address cannot be resolved to a name, and you end up getting only the client's IP address, exactly as if you had called the `getRemoteAddress` method. Services available on the Internet let you resolve an IP address to the country where the system resides, but you might have to pay for a reliable service.

Caution You cannot mix methods that handle parameters with methods that handle the request content or methods that access the request content in different ways. For example, if you execute any two of the methods `request.getParameter`, `request.getReader`, and `request.getInputStream` when handling a request, the second one you execute will fail.

Example: Listing the Headers

Create new jsp in the same test Project folder created in Chapter 1 and copy the following code or place it in the WebContent folder. Listing 2-5 shows code that displays the request headers.

Listing 2-5. req_headers.jsp

```
<%@page language="java" contentType="text/html"%>
<%@page import="java.util.*"%>
<html><head><title>Request Headers</title></head><body>
<%
    Enumeration headers = request.getHeaderNames();
    int kh = 0;
    while (headers.hasMoreElements()) {
        String hName = (String)headers.nextElement();
        out.println("----- " + hName);
        Enumeration hValues = request.getHeaders(hName);
        while (hValues.hasMoreElements()) {
            out.println("<br/>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;" + hValues.nextElement());
        }
        out.println("<br/>");
    }
    %>
</body></html>
```

At last, start Tomcat in Eclipse and type in a browser the URL

http://localhost:8080/test/req_headers.jsp

Figures 2-1 and 2-2 show the request headers generated, respectively, by Chrome and Firefox. Interesting, aren't they?